IKONEX ACADEMY SMS

ORAL INTERVIEW PREPARATION GUIDE

Theo Korir · 2025

This document covers everything you need for the oral interview: architecture explanations, design decisions, challenges and how you solved them, and which sections of code to walk through. Study the Q&A answers until you can say them naturally — not recite them.

1. Architecture & Design Decisions

Q: Walk me through your architecture.

It's a full-stack Next.js 15 application using the App Router. The frontend is React with TypeScript, and the backend is Next.js Route Handlers — serverless functions that run on Node.js. The database is libSQL, which is SQLite-compatible, connected to Turso in production. The entire thing deploys as a single unit to Vercel — no separate backend server to manage, no separate deployment pipeline.

Tip: Draw a simple box diagram: Browser → Vercel (Next.js) → Turso. Keep it visual.

Q: Why Next.js instead of a separate React frontend and Node backend?

For a project at this scope, a separate backend adds deployment complexity without adding value. Next.js API routes give me a proper REST API that runs on Node.js and deploys as serverless functions on Vercel. One repo, one deploy, one set of environment variables — and the same TypeScript types can be shared across frontend and API if needed.

Q: Why libSQL / Turso instead of PostgreSQL or MySQL as recommended?

The assessment recommended MySQL or PostgreSQL, and I considered both. I chose libSQL with Turso because the SQL dialect is identical to SQLite — my development database is a local file, no Docker or local Postgres install needed. In production, Turso is production-grade cloud SQLite with the same driver. The tradeoff is that SQLite lacks some PostgreSQL features like JSONB and full-text search — but none of those are needed here. The dev-to-prod parity with zero code changes was the deciding factor.

Tip: Be upfront about the tradeoff. Interviewers respect honesty over over-selling.

Q: Explain your database schema.

Six core tables. ClassStream holds streams like Form 1A. Student belongs to one stream via a foreign key. Subject is independent. ClassStreamSubject is a junction table for the many-to-many relationship between streams and subjects. Score records exam and CAT marks per student per subject per term and year, with a unique constraint to prevent duplicates. GradingScale stores the KCSE grade bands and is seeded automatically on first run.

Q: Why a junction table for ClassStreamSubject?

Because streams and subjects have a many-to-many relationship. Form 1A and Form 1B can both offer Mathematics — I don't want to duplicate the subject record. A junction table handles that cleanly. It also means a stream can have a subject added or removed without touching the core Subject or ClassStream tables.

Schema at a glance:

Table	Purpose	Key constraint
ClassStream	Streams e.g. Form 1A	name UNIQUE
Student	Student records	FK → ClassStream
Subject	School subjects	name UNIQUE, code UNIQUE
ClassStreamSubject	Which subjects belong to a stream	UNIQUE(streamId, subjectId)
Score	Exam + CAT marks per student/subject	UNIQUE(studentId, subjectId, term, year)
GradingScale	KCSE grade bands A → E	Seeded on first run

2. How Key Features Work

Q: How does results processing work?

The /api/results route fetches all students in a stream, all subjects assigned to that stream, and all scores for the selected term and year. It loops through each student, sums their marks, calculates their average, looks up the grade from GradingScale, then does two sorting passes: one per subject to assign subject positions, and one overall to assign class rank. All server-side on each request — no caching layer needed at this scale.

Q: How do you prevent duplicate score entries?

Two layers. First, the Score table has a UNIQUE constraint on (studentId, subjectId, term, year) — the database enforces it at the lowest level. Second, the API detects whether a score exists for that combination and switches between POST and PUT accordingly. On the frontend, existing records are marked 'Recorded' so the user knows before they save.

Q: How does PDF generation work?

jsPDF is a browser-only library — it can't run in a Next.js server route. So the download button triggers a dynamic import of jsPDF in the browser, builds the PDF from results data already loaded in React state, and downloads it directly. No server involvement, no extra API call. The dynamic import also means the library isn't bundled into the initial page load — it's only fetched when the user clicks download.

3. Challenges & How You Solved Them

initDB() running on every request

Problem: Every API request was triggering 8+ CREATE TABLE statements against Turso over HTTP. It worked, but it was slow and wasteful.

Solution: Introduced a module-level singleton — a boolean flag and a shared Promise. Init runs exactly once per server process. Concurrent requests on cold start wait for the same Promise rather than racing each other.

executeMultiple() risk on Turso

Problem: Batching all CREATE TABLE statements in one executeMultiple() call works locally but can swallow errors silently on Turso's HTTP transport if one statement fails.

Solution: Split into individual execute() calls. Each failure surfaces immediately with a clear error rather than the batch silently succeeding or partially failing.

PDF generation in a server-rendered environment

Problem: jsPDF is a browser library. Importing it in a Next.js API route causes a build error because it references browser globals like window.

Solution: Moved PDF generation entirely to the client side using a dynamic import inside the download button handler. The library is only loaded when needed, and it runs in the browser where window is available.

Score entry for a full class — UX problem

Problem: Entering scores one student at a time would be impractical for a teacher with 40+ students.

Solution: Built a bulk entry table — the teacher selects stream and subject, and all students appear as a single editable grid. One 'Save All' button fires a loop of POST/PUT calls and reports how many succeeded.

4. Key Code Sections to Walk Through

1. lib/db.ts — Singleton init pattern

Point to the initialised boolean and initialisingPromise. Explain that without it, 10 concurrent requests on cold start would each race to create the tables simultaneously, causing duplicate-creation errors or race conditions.

```
let initialised = false;
let initialisingPromise: Promise | null = null;

export async function initDB() {
  if (initialised) return;
  if (initialisingPromise) return initialisingPromise;
  initialisingPromise = (async () => {
    // ... create tables ...
    initialised = true;
  })();
  return initialisingPromise;
}
```

2. app/api/results/route.ts — Results processing

The most complex route. Walk through: fetch students → fetch subjects → fetch scores → loop students → calculate totals → look up grades → sort for subject positions → sort for class rank. Two separate sort passes is the key point.

```
// 1. Fetch all data
const students = await db.execute(...);
const subjects  = await db.execute(...);
```

```
const scores    = await db.execute(...);

// 2. Per-student aggregation
const results = students.map(student => {
  const studentScores = allScores.filter(s => s.studentId === student.id);
  const totalMarks = studentScores.reduce((sum, s) => sum + s.examScore + s.catScore, 0);
  const { grade } = getGrade(totalMarks / count, gradingScales);
  return { ...student, totalMarks, grade, position: 0 };
});

// 3. Rank by total marks
results.sort((a, b) => b.totalMarks - a.totalMarks)
       .forEach((r, i) => r.position = i + 1);
```

3. app/scores/page.tsx — Bulk score entry with POST/PUT detection

Show how the page loads existing scores into state, marks them as 'Recorded', and on save switches between POST (new) and PUT (update). This is the duplicate prevention logic from the frontend side.

```
const isExisting = existing.find(e => e.studentId === student.id);
const method = isExisting ? 'PUT' : 'POST';

const r = await fetch('/api/scores', {
  method,
  body: JSON.stringify({ studentId, subjectId, examScore, catScore, term, year })
});

if (r.ok) saved++; else errors++;
```

4. app/results/page.tsx — Dynamic PDF import

Show the dynamic import inside the click handler. Explain why: jsPDF references browser globals, so it can't be imported at the top of the file in a Next.js environment that also does server-side rendering.

```
const downloadPDF = async (studentData) => {
  // Dynamic import = loads only in browser, only when clicked
  const { default: jsPDF }    = await import('jspdf');
  const { default: autoTable } = await import('jspdf-autotable');

  const doc = new jsPDF();
  autoTable(doc, {
    head: [['Subject', 'Exam', 'CAT', 'Total', 'Grade']],
    body: studentData.subjectResults.map(sr => [...]),
  });
  doc.save(`ReportCard_${studentData.admNo}.pdf`);
};
```

5. Grading Scale (KCSE) — Know This Cold

Grade	Min	Max	Points
A	75	100	12
A-	70	74	11

B+	65	69	10
B	60	64	9
B-	55	59	8
C+	50	54	7
C	45	49	6
C-	40	44	5
D+	35	39	4
D	30	34	3
D-	25	29	2
E	0	24	1

6. Last-Minute Reminders

- Open your code in VS Code before the interview. Know where every file is.
- Be able to draw the database schema on a whiteboard from memory.
- If you don't know something, say so clearly — then reason through it aloud.
- Don't say 'the AI helped me' without being able to walk through the code line by line.
- Explain tradeoffs honestly: SQLite vs Postgres, why you chose what you chose.
- Have the live URL open and ready to demo before they ask.
- When walking through code, narrate what you're looking for before you find it.